GNSS M9N — RP2040 Architecture

Module: u-blox M9N GNSS receiver interfaced to a Raspberry Pi Pico (RP2040)
Framework: Arduino (PlatformIO, Mbed variant)
Last Updated: May 29, 2026

---

1. Overview

The M9N is a concurrent GNSS receiver (GPS / GLONASS / Galileo / BeiDou). It exposes two independent communication ports used simultaneously:

Port	Direction	Purpose
I2C (400 kHz)	M9N → RP2040	Stream NMEA sentences (GGA, RMC, GSA)
UART (38400 baud)	RP2040 → M9N	Send UBX configuration commands

NMEA data flows in on I2C; configuration commands go out on UART. The two ports operate independently — configuring via UART does not disturb the NMEA stream on I2C.

---

2. Hardware Connection

Pin Map

```
┌─────────┬──────────┬──────────────────────────────────────────┐
│ GPIO    │ Dir MCU  │ Function                                 │
├─────────┼──────────┼──────────────────────────────────────────┤
│ GPIO4   │ I/O      │ I2C0 SDA — NMEA data stream from M9N     │
│ GPIO5   │ I/O      │ I2C0 SCL — I2C clock                     │
│ GPIO16  │ OUT      │ UART1 TX  — UBX config commands to M9N   │
│ GPIO17  │ IN       │ UART1 RX  — (unused; reserved for ACK)   │
└─────────┴──────────┴──────────────────────────────────────────┘
```

Wiring Diagram

```
RP2040                          u-blox M9N
──────                          ──────────
3.3 V ──────────────────────────── VCC
GND   ──────────────────────────── GND

GPIO4 (SDA) ←──── 4k7 ── 3V3   SDA    (I2C0, 400 kHz)
GPIO5 (SCL) ──── 4k7 ─── 3V3   SCL
```

```
GPIO16 (TX1) ──────────────────── UART_RX  (UBX commands, 38400 baud)
GPIO17 (RX1) ←──────────────────── UART_TX  (reserved)
```

I2C address: **0x42** (M9N default, not configurable in hardware).

---

3. Communication Protocols

3.1 I2C — Reading NMEA Data

The M9N maintains an internal byte-FIFO. Registers **0xFD** (MSB) and **0xFE** (LSB) expose the number of bytes waiting. Unused positions in the FIFO are filled with **0xFF** padding.

Read sequence (executed each `update()` call):

```
1. Write register pointer 0xFD to M9N (repeated-start)
2. Read 2 bytes → available = (byte[0] << 8) | byte[1]
3. While available > 0:
     chunk = min(available, 255)
     requestFrom(0x42, chunk)
     feed each byte to NMEA state machine
     available -= chunk
```

3.2 UART — Sending UBX Configuration

UBX is u-blox's binary protocol for configuration. `begin()` sends one UBX-CFG-VALSET frame to set the dynamic model to **AIRBORNE_2G** (UAV / drone profile, up to 2g acceleration).

UBX-CFG-VALSET frame (17 bytes):

```
Offset  Bytes  Value        Description
──────  ─────  ──────────   ──────────────────────────────────
0       2      B5 62        Sync chars
2       2      06 8A        Class / ID = UBX-CFG-VALSET
4       2      09 00        Payload length = 9
6       1      00           Version
7       1      07           Layers: RAM (b0) | BBR (b1) | Flash (b2)
8       2      00 00        Reserved
10      4      21 00 11 20  Key = CFG-NAVSPG-DYNMODEL (0x20110021, LE)
14      1      <model>      Dynamic model value (0x07 = AIRBORNE_2G)
15      2      CK_A CK_B    Fletcher-8 checksum over bytes [2..14]
```

Available dynamic models (defined in `gnss.h`):

Constant	Value	Use case
`UBX_DYNMODEL_PORTABLE`	0	Default

Constant	Value	Use case
`UBX_DYNMODEL_STATIONARY`	2	Fixed installation
`UBX_DYNMODEL_AIRBORNE_1G`	6	High-altitude balloon
`UBX_DYNMODEL_AIRBORNE_2G`	7	UAV / drone ← **active**
`UBX_DYNMODEL_AIRBORNE_4G`	8	Rocket / aggressive flight

---

4. Firmware Architecture

4.1 Files

```
include/gnss.h      — GPSData struct, GNSS class declaration, UBX model constants
src/gnss.cpp        — GNSS class implementation + NMEA parsers (file-scope static)
include/config.h    — Pin assignments and bus parameters
```

4.2 GPSData Struct

```
struct GPSData {
  double   latitude;    // decimal degrees, + N / − S
  double   longitude;   // decimal degrees, + E / − W
  double   altitudeM;   // metres above MSL

  uint8_t  fixQuality;  // GGA field 6: 0=none, 1=GPS, 2=DGPS …
  uint8_t  fixType;     // GSA field 2: 1=none, 2=2D, 3=3D
  uint8_t  satellites;  // satellites in use

  double   speedKnots;
  double   courseDeg;

  uint8_t  hour, minute, second;   // UTC time from GGA/RMC
  uint8_t  day, month;
  uint16_t year;                   // from RMC

  bool     valid;   // true when fixQuality > 0 (GGA) or status == 'A' (RMC)
};
```

4.3 GNSS Class Interface

```
class GNSS {
public:
  bool            begin();        // I2C check, UART init, set AIRBORNE_4G model
  void            update();       // drain M9N I2C FIFO, parse NMEA
  const GPSData&  getData() const;
```

```
    bool            hasNewFix() const;  // true for one update() cycle after new fix
};
```

4.4 NMEA Parser — Static Scope

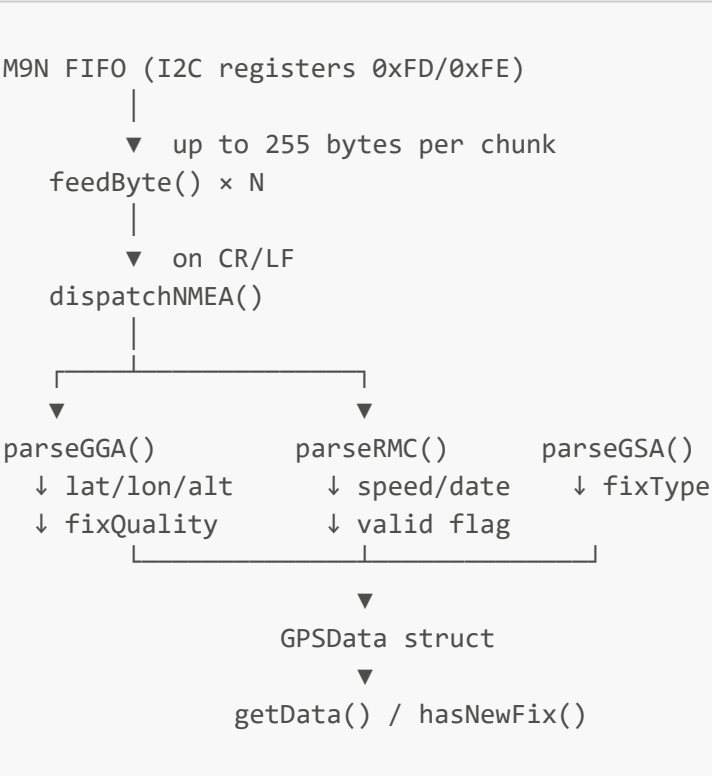
All parsing helpers are **file-scope static** (not class members). This keeps the class interface clean while allowing the parsers to write directly into the active `GPSData` instance via module-level pointers set in `begin()`.

```
feedByte(b)
  │
  ├─ 0xFF → discard (M9N padding)
  ├─ '$'  → start new sentence buffer
  ├─ CR/LF → sentence complete → dispatchNMEA()
  └─ else → append to nmeaBuf[128]

dispatchNMEA(sentence)
  │
  ├─ nmeaChecksum() → drop if bad XOR
  ├─ "**GGA" → parseGGA()  — position, time, fix quality, altitude
  ├─ "**RMC" → parseRMC()  — position, time, speed, course, date, validity
  └─ "**GSA" → parseGSA()  — fix type (1/2/3)
```

Talker-agnostic matching: the parser skips the 2-char talker prefix (`GP`, `GN`, `GL`, `GA`) and matches only on the 3-char sentence type (`GGA`, `RMC`, `GSA`).

4.5 Data Flow



5. Configuration (`include/config.h`)

```
// I2C0 — GNSS NMEA data
#define I2C_SDA_PIN       4
#define I2C_SCL_PIN       5
#define I2C_CLOCK_HZ      400000
#define GNSS_I2C_ADDR     0x42

// UART1 — UBX config commands
#define GNSS_UART_TX_PIN  16
#define GNSS_UART_RX_PIN  17
#define GNSS_UART_BAUD    38400
```

---

6. Build System

```
[env:pico]
platform        = raspberrypi
board           = pico
framework       = arduino
upload_protocol = picotool
```

No external GNSS library is used. The NMEA parser and UBX framing are implemented directly in `gnss.cpp`.

```
pio run                    # compile
pio run --target upload    # compile + upload (BOOTSEL mode)
pio device monitor -b 115200
```

---

7. Typical Usage in `main.cpp`

```
#include "gnss.h"
#include "config.h"
#include <Wire.h>

GNSS gnss;

void setup() {
  Serial.begin(115200);
  Wire.setSDA(I2C_SDA_PIN);
  Wire.setSCL(I2C_SCL_PIN);
  Wire.begin();
  Wire.setClock(I2C_CLOCK_HZ);

  if (!gnss.begin()) {
```

```
    Serial.println("M9N not found on I2C");
    while (true);
  }
}

void loop() {
  gnss.update();
  if (gnss.hasNewFix()) {
    const GPSData& d = gnss.getData();
    Serial.printf("%.6f, %.6f  alt=%.1fm  sats=%d\n",
                  d.latitude, d.longitude, d.altitudeM, d.satellites);
  }
  delay(10);
}
```

---

8. Known Constraints

- **I2C bus shared**: Wire (I2C0) is shared with anything else on GPIO4/5. The M9N does not support clock-stretching beyond what the RP2040 I2C peripheral tolerates; keep other devices off this bus or schedule reads carefully.
- **I2C blocking**: A full FIFO drain (~1 KB at 400 kHz) takes ~25 ms.
- **UBX ACK not checked**: `sendUBX()` writes to UART and returns immediately. The M9N will ACK or NAK on UART_TX but `GPIO17 RX` is not read — add a response parser if config confirmation is needed.
- **0xFF padding**: The M9N pads its I2C output buffer with `0xFF` when idle. `feedByte()` silently discards these; they are not NMEA data.